Copyright © 2023. Bentham Science Publishers. All rights reserved.

File Handling

Krishna Kumar Mohbey[1], **Malika Acharya**[1]

[1] Computer Science, Central University of Rajasthan, India

Abstract

From the previous chapters, we anticipate that the readers have garnered enough working knowledge of Python and its elementary concepts. In this chapter, we introduce to the user the concept of files. These are one of the major built-in object types in Python. We can create, call, work, and close the files using several functions as enunciated below. One of the primary tasks of files is method exporting and common file-processing tasks such as input and output display to external files, flush buffers, *etc*.

Keywords: Buffering, Directory path in OS, Storage in OS.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

INTRODUCTION

When information must be stored in a file in an unalterable manner, file management is essential. A file is a designated location on the disc where data is kept that is important to the purpose of the file. Once the program has been closed, we will be able to retrieve the information that was previously stored. The concept of file management has been adapted and implemented in various other languages. However, doing so can be difficult or time-consuming, depending on the language.

However, unlike other Python ideas, this one is not complicated and can be understood easily. You will learn various theories and procedures about the management of files in this chapter. In addition, we have illustrated a variety of file actions by using examples and programs to illustrate our points.

FILE

A collection of bytes that may be used to store information is called a file. This data is structured in a specific format, which might be anything from a straightforward text file to an intricate program executable, depending on its complexity. In the end, these byte files are changed into binary, consisting of 1s and 0s so that the computer can process them more quickly. Much like many other programming languages, Python can handle files, allowing users to read, write, and execute many other file-related operations. This functionality is known as file handling. It is essential to understand that Python processes files differently based

on whether they contain text or binary data. Each line of code is made up of a string of individual characters that, when combined, make up a text file. A special character terminates each line in a file referred to as the EOL (End of Line) character. Examples of EOL characters include the comma and the newline character. It signals to the interpreter that the line being read has finished and that a new line has begun. On most of today's file systems, files are broken up into three distinct parts:

- *Header:* details about the file's contents (file name, size, type, and so on).
- *Data:* the file's contents as written by the author or writer.
- *End of file:* a unique character that denotes the file's termination.

In programming, it is possible that a specific piece of input data must be created more than once. Occasionally, it is insufficient only to display data on the console. Large amounts of data may be

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

presented. The console can only display a certain amount of data; as memory is volatile, it is difficult to restore programmatically created data frequently. The local file system, which is volatile and always available, is where we can store things if we need to. The file-handling capabilities of Python must be used for this. We can use our Python application to create, modify, read, and destroy files on the local file system, thanks to file management in Python. When accessing a file on an operating system, a file path is necessary. A string indicates the location of a file called a file path. There are three main sections in it:

- **Directory Path:** the location of a file or folder on a file system, separated by a forward slash (/) in Linux or Unix or a backslash (\) in Windows.
- **File Name:** the file's real name.
- **Extension:** it defines the file type.

Let's say you needed to open the T1.txt file, and the position you were in now was the same as the **location**. You must first travel to the **Location** folder, then to the **Folder1** directory, and then to the **T1.txt** file to access the file. The path of this file is "***Location/Folder1/T1.txt.***"

Open() Function

It must be open first to perform reading and writing operations on a file. To open a file in Python, the user must first create a file object associated with a physical file. In addition, the ***open()*** function is used to open a file in Python. Python's open() function takes two arguments: the file name and the access mode. The function returns a file object, which can be used for reading, writing, and other operations. The below syntax is used to open a file in Python.

Opening a file is a prerequisite for performing any action on it, including reading and writing. To access the contents of a file using Python, the user must first build a file object corresponding to the file's actual location. In Python, opening a file is also accomplished with the help of the ***open()*** function. Python's ***open()*** function requires two pieces of information before it can be called: the *file name* and the *access mode*. The method will return a file object that may be utilized for various tasks like reading, writing, and others. In Python, opening a file is accomplished with the syntax that is presented below:

Syntax:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
File_object_Name = open(<file_name>, <access_mode>, <buffering>)
```

Various modes, such as read, write, and append, are available for accessing the files. The access mode to open a file is defined in Table 1.

Table 1 **File Access Modes.**

R	The file is opened in read-only mode. The file pointer is present at the start. If no access mode is defined, the file is opened in this mode by default.
rb	It converts the binary file into a read-only mode. The file pointer is present at the start of the file.
r+	It opens the file for both reading and writing. The file pointer is present at the start of the file.
rb+	It opens the file in binary format for reading and writing. The file pointer is present at the start of the file.
W	It only allows you to write to the file. If a file with the same name already exists, it is overwritten; otherwise, it is created. The file pointer is present at the start of the file.
wb	It opens the file so it can only be written in binary format. If the file already exists, it is overwritten; otherwise, it generates a new one. The file pointer is present at the start of the file.
w+	It opens the file for both writing and reading. It differs from r+ in that it overwrites the previous file if one exists, while r+ leaves the previously written file alone. If no such file exists, it creates one. The file pointer is present at the start of the file.
wb+	It opens the file in binary format for both writing and reading. The file pointer is present at the start of the file.
A	The file is opened in append mode. If there is one, the file pointer is at the end of the previously written file. If no file with the same name exists, it creates a new one.
ab	It opens the file in binary format in append mode. The pointer is at the end of the file that was previously written. If no file of the same name remains, it produces a new binary file.
a+	It opens a file for both appending and reading. If a file exists, the file pointer stays at the end of it. If no file with the same name exists, it produces a new one.
ab+	It opens a binary file for appending and reading. The file pointer is already at the file's end.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
1. obj = open("Myfile.txt","r") #Opening Myfile.txt in read mode
2.
3. if obj:
4.     print (File opened successfully.")
```

Output:

```
File opened successfully.
```

Close() Function

Since Python allows us to do any operation on the file by utilizing the file system that is presently open, it is recommended practice to close the file after all processes have been finished. After we have finished working with the file in every possible way, we will shut it by calling the ***close()*** function. Any data that has not yet been written is discarded when a file object's close() function is invoked.

Syntax:

```
Object_Name.close()
```

Example:

```
1. obj = open("Myfile.txt","r") #Opening Myfile.txt in read mode
2.
3. If obj:
4.     print (File opened successfully.")
5. obj.close()        # file will be closed
```

Output:

```
File opened successfully.
```

Write() Function

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

A string is written to a file using the ***write()*** function. To write text to a file, we must first open it with the open method and one of the access modes mentioned below:

- **w:** If the file already exists, it will be overwritten. The file pointer is located at the start of the file.
- **a:** The file will be appended to the current one. The file pointer is at the file's end. If no such file exists, it creates one.

Example:

```
# Open the file in write mode
file1 = open("file1.txt","w")
#Write contents in file
file1.write("Hello \n")
file1.write("My Python Programming book \n")
file1.write("Python is easy \n")
file1.write("Python is developers' choice \n")
#Close the file
file1.close()
```

Output:

You can see the file1 created and can open it to see the contents.

```
Hello
My Python Programming book
Python is easy
Python is developers choice
```

Writelines() Method

The ***writelines()*** method is a Python built-in method for writing a list of strings or list items to a file.

Syntax:

```
fileobject.writelines(list_of_items)
```

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
#Open the file in write mode
f1 = open("f1.txt","w")
#Add the list of contents to be added
l1 = ["My first Python book \n","Python is simple \n","File handling is easy\n"]
f1.write("Hello \n")
f1.writelines(l1)
f1.close()
#To read the contents of the file, open it
f1 = open("f1.txt","r+")
    #Select to read it
print(f1.read())
print()
```

Output:

```
Hello
My first Python book
Python is simple
File handling is easy
```

Writing Numbers to a File

Python reads files as strings by default, and the ***write()*** function expects only to write strings. If we want to write numbers to a file, we'll need to use the ***str()*** function to cast them as strings.

Example:

```
a = range(1,30)
f2 = "f2.txt"
# Open file to write
f2= open(f2, 'w')
for num in a:
  f2.write(str(num) + " ")
f2.close() #Close the file
#To read the contents of the file, open it
f2 = open("f2.txt","r+")
#Select to read it
print(f2.read())
print()
```

Output:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Read() Method

In Python, there are several ways to read a text file. The ***read()*** method can be used to retrieve a string that includes all the characters in a file.

Syntax:

```
File_Object.read()
```

Before reading a file, it must be opened in '**r**' (read) mode.

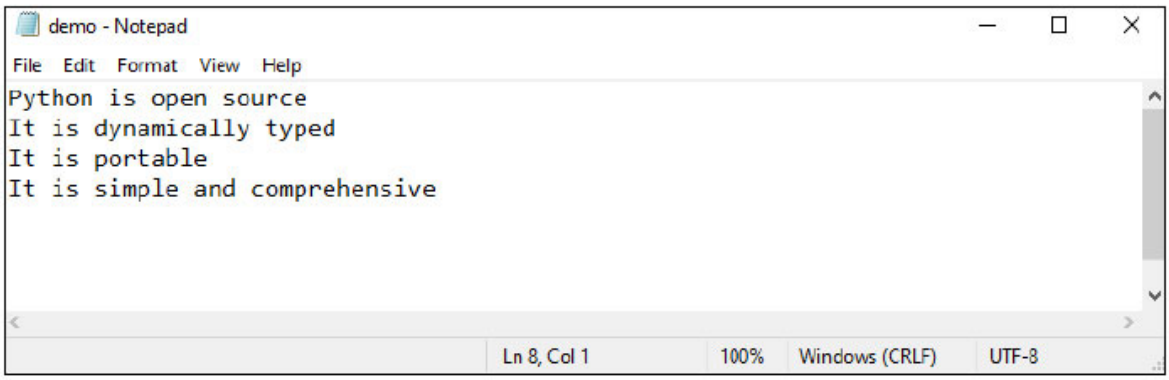


Fig. (1))

Contents of demo.txt.

NOTE: For demonstrating the use of read(), readline() and readlines() commands, we will consider Fig. (**1**) as the specimen. This figure shows the file containing demo.txt that is retrieved using read commands.

Example:

```
1. #Open the file in read mode
2. f3 = open("demo.txt","r+")
3. print(f3.read())
4. print()
5. f3.close() # change the mode
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
Python is open source
It is dynamically typed
It is portable
It is simple and comprehensive
```

Another technique to read a text is limiting yourself to a particular number of characters. For instance, the interpreter will read the first twenty characters of the data that has been saved and will then return it as a string by utilizing the following code:

Example:

```
1. #Open the file in read mode
2. f4 = open("demo.txt","r+")
3. #Select to read it
4. print(f4.read(16))
5. print()
6. f4.close() # change the mode
```

Output:

```
Python is open s
```

Readline() Method

Python's ***readline()*** function makes it simple to read lines of text from a file in a sequential order. Since the ***readline()*** function reads lines from the file beginning with the first one, we will obtain the first three lines of the file if we call it three times, and it will start from the beginning. Every time it is invoked, this function will provide the calling code with a character string that contains a single line of data read from the file.

Example:

```
1. #Open the file in read mode
2. f5 = open("demo.txt","r+")
3. #Select to read it
4. print(f5.readline())
5. print()
6. f5.close()# change the mode
```

Output:

```
Python is open source
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

To retrieve only a specified number of characters, say four, specify the number of the line along with readline().

Example:

```
1. #Open the file in read mode
2. f6 = open("demo.txt","r+")
3. #Select to read it
4. print(f6.readline(4))
5. print()
6. f6.close() # change the mode
```

Output:

```
Pyth
```

Readlines() Method()

If we wanted to get back every line in the file separated adequately. The same function will be used but in a different format. The ***FileObject.readlines()*** method is used to accomplish this. It returns a list of lines up to the end of the file (EOF).

Example:

```
1. #Open the file in read mode
2. f7 = open("demo.txt","r+")
3. #Select to read it
4. print(f7.readlines())
5. print()
6. f7.close() # change the mode
```

Output:

```
['Python is open source\n', 'It is dynamically typed\n', 'It is portable\n', 'It is simple and comprehensive']
```

Reading Contents with a Loop

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

The loop-over approach may be utilized to read all of the lines from a file in a quicker and more memory-efficient manner than any other method. One of the benefits of utilizing this strategy is that the code that corresponds to it is straightforward to understand. Fig. (2) shows the contents of file **demo2.txt** that we will retrieve using a loop.

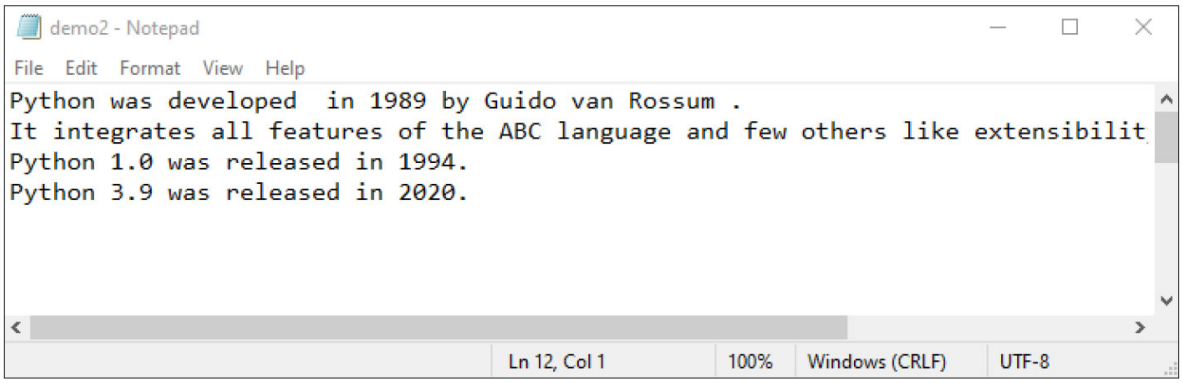


Fig. (2))

Contents of demo2.txt.

Example:

```
#Open the file in read mode
f8 = open("demo2.txt", "r")
#Select to read it
for x in f8:
    print(x)
f8.close() # change the mode
```

Output:

```
Python was developed  in 1989 by Guido van Rossum .

It integrates all features of the ABC language and few others like extensibility and exception handling.

Python 1.0 was released in 1994.

Python 3.9 was released in 2020.
```

Creating a New File

With the help of the ***open()*** function, you can create a new file by using one of the following access modes:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

- **x:** It creates a new file with the name you specify. It throws an error if a file with the same name already exists.
- **a:** If no such file exists, it creates a new one with the specified name. If the file with the given name already exists, it appends the content to it.
- **w:** If no such file exists, it creates a new one with the specified name. If a file exists, it overwrites.

Example:

```
1. #Creating a new empty file
2. fileobj= open ("file1.txt", "x")
3. #close this file
4. fileobj.close()
```

The above code will create a new file in the current working directory. If you want your file to be created at an absolute path on the disc, you can also give it a complete path.

Example: Creating a new file with an existing name

```
1. #Creating a new file, but myfile1.txt is already exist
2. fileobj= open ("file1.txt", "x")
```

Output:

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    fileobj= open("file1.txt", "x")
FileExistsError: [Errno 17] File exists: 'file1.txt'
```

In the above example, we tried to create a new file file1.txt. However, this file already exists in the same directory. We get FileExistsError.

File Object Attributes

You can get different file-related details once a file is opened and you have one file object. The following is a list of all file object attributes:

- **closed** – It is a bool value that indicates the current state of the file object.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

- **encoding** - The encoding details of the file.
- **mode** – It returns the file mode of the file in which the file is opened.
- **name** – It returns file name.

Example:

```
1. import os.path
2. f10 = open("demo2.txt", 'a+')
3. #Specifying the required attributes
4. print("Status:",f10.closed)
5. print("Encoding style:",f10.encoding)
6. print("Mode:", f10.mode)
7. print("Name:",f10.name)
```

Output:

```
Status: False
Encoding style: UTF-8
Mode: a+
Name: demo2.txt
```

File Positions

When we use Python to open a file for reading, we are provided with a file handle that includes a reference to the beginning of the file. When we read from the file, the pointer will always point to the end of the reading, and the beginning of the following read will be at that location. The following methods are used, respectively, to acquire the current file position and alter the current file position:

- The **tell()** method returns the current file position; in other words, the next read or write will occur at that many bytes from the file's beginning.
- The **seek(offset[, from])** method is used to adjust the file's current location. The number of bytes to be moved is defined by the offset statement. The from statement determines the starting point for the bytes to be transferred from.

If from is set to **0** in the ***seek()*** function, the file's beginning will be used as the reference position, **1** the current position will be used as the reference position, and **2** the file's end will be used as a reference position.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
1. # Open the file in write mode
2. f_1 = open("demo3.txt","w")
3. f_1.write("Python tools can be classified into two categories, general-
purpose tools, and functional tools.  \n")
4. f_1.writelines("Python has full-fledged support for a vast variety of built-in objects -
types.  \n")
5. f_1.writelines("Python considers memory as the private heap. \n")
6. f_1.writelines("Python community support is quite active and responsive  \n")
7. f_1.close() #to change file access modes
8. f_1= open("demo3.txt", "rb")
9. print("Position of pointer:", f_1.tell())
10.    #Reading first line
11.    l1= f_1.readline()
12.    print(l1)
13.    print("Position of pointer:", f_1.tell())
14.    #Reading second line
15.    l2= f_1.readline()
16.    print(l2)
17.    print("Position of pointer:", f_1.tell())
18.    #Reading third line
19.    l3= f_1.readline()
20.    print(l3)
21.    print("Position of pointer:", f_1.tell())
22.    #Reading fourth line
23.    l4= f_1.readline()
24.    print(l4)
25.    print("Position of pointer:", f_1.tell())
26.
27.    #changing the pointer's position
28.    #moving pointer 15 positions back from end of the file
29.    print("changed pointer position:", f_1.seek(-15,2))
30.    f_1.close()
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
Position of pointer: 0
Python tools can be classified into two categories, general-purpose tools, and functional tools.

Position of pointer: 99
Python has full-fledged support for a vast variety of built-in objects -types.

Position of pointer: 180
Python considers memory as the private heap.

Position of pointer: 226
Python community support is quite active and responsive

Position of pointer: 284
changed pointer position: 269
```

File Rename

Python's **OS** module offers several ways to carry out various file-processing tasks, one of which is renaming files. To utilize this module, you must first import it and then use any linked functions. The renaming of files and folders works in exactly the same way. The ***os.rename()*** method is used whenever a file's name must be changed.

Syntax:

```
os.rename(Previous_file_name, new_file_name)
```

Example:

```
1. import os
2. print ('original file',os.listdir(os.getcwd()))
3. old_name = r"/content/f1.txt"
4. new_name = r"/content/demonstration.txt"
5. # Renaming the file
6. os.rename(old_name, new_name)
7. print ('Rename:',os.listdir(os.getcwd()))
```

Output:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
original file ['.config', 'f1.txt', 'f2.txt', 'demo3.bin', 'myfile.txt', 'demo.txt', 'demo2.txt', 'sample_data']

Rename: ['.config', 'demonstration.txt', 'f2.txt', 'demo3.bin', 'myfile.txt', 'demo.txt', 'demo2.txt', 'sample_data']
```

Deleting a File

os.remove() function is used to delete a specific file.

Syntax:

```
os.remove(file_name)
```

Example:

```
1. import os
2. print ('List of files',os.listdir(os.getcwd()))
3. # Specifying the path of file to be removed
4. os.remove("/content/f2.txt")
5. print ('Updated list:',os.listdir(os.getcwd()))
```

Output:

```
List of files ['.config', 'demonstration.txt', 'f1.txt', 'f2.txt', 'demo.txt', 'demo2.txt', 'sample_data']

Updated list: ['.config', 'demonstration.txt', 'f1.txt', 'demo.txt', 'demo2.txt', 'sample_data']
```

Binary Files

Text and binary files are the two types of files that may be used with Python. The bulk of the files that can be found on our computers are of the binary file type. Utilizing Python's features makes working with binary files straightforward. Binary files make use of strings that are in the form of bytes. Reading binary data from a file indicates that you will receive an object of type byte in return. When working with binary files, we are required to make use of the identical modes (as before), but with the letter ***'b'*** added to the end in a way that will allow Python to understand that we are working with binary files. To write the binary file, the ***'wb'*** mode is utilized. Similarly, the ***'rb'*** and ***'ab'*** modes are utilized in binary files to read and add data, respectively.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
num=[2,4,8,16,32,64,128]
arr=bytes(num)
print(arr)
f=open("demo3.bin","wb")
f.write(arr)
f.close()
#open binary file
f = open("demo3.bin", 'rb')
content = f.read()
content
```

Output:

```
b'\x02\x04\x08\x10 @\x80'
```

Directory Operations

A directory is a container for all connected files, folders, and documents. The OS module in Python provides tools for interacting with the operating system. The operating system-specific operations for controlling processes, files, file descriptors, directories, and other low-level OS features are accessible through this module.

Current Working Directory

getcwd() returns the current working directory's path. This is where the operating system converts a relative file name to an absolute file name.

Syntax:

```
#importing os module
import os
#print the current working directory
print (os.getcwd())
```

Output:

```
/home/user
```

List of Directories

We can get a directory list from a specific location. It would help if you used the ***listdir(location)*** function. The function will return a list of strings containing the names of the directories in the

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

specified location if you pass it to the location. The simple **listdir()** function will return the contents of the current directory.

Example:

```
1. import os
2. data = os.listdir() #list of contents from current direcotry
3. print (data)
4.
5. print (os.listdir('/usr')) # list of contents from  usr direcotry
```

Output:

```
['main.py']
['games', 'local', 'include', 'share', 'lib', 'bin', 'src', 'sbin']
```

Creating a Directory

The ***os.mkdir()*** method is used to create a directory. Let's make a new directory called "**mydir**" The os.listdir() method will print a list of directories along the path.

Example:

```
1. import os
2.  #create directory
3. os.mkdir("dir1")
4. print (os.listdir()) # list of contents
```

Output:

```
['dir1', 'main.py']
```

Change Directory

To change the directory, we must first import the os module and then use the ***os.chdir()*** method to change our program's base path.

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
import os
#change directory
os.chdir('/Users/folder1/')

#print current working directory
print (os.getcwd())
```

Output:

```
/Users/folder1
```

Renaming a Directory

The ***os.rename()*** method allows you to rename a folder from one name to another.

Example:

```
import os
#rename directory
os.mkdir('mydir1')
print (os.listdir(os.getcwd()))
#rename
os.rename("mydir1","mydir2")
print (os.listdir(os.getcwd()))
```

Output:

```
['mydir1', 'main.py']
['mydir2', 'main.py']
```

Delete a Directory

The ***rmdir()*** function is used to remove an already empty directory. The directory will not be removed if it is not empty.

Example:

```
import os
os.mkdir('mydir1')
print ('Before deletion:',os.listdir(os.getcwd()))

#delete directory
os.rmdir("mydir1")
print ('After deletion:',os.listdir(os.getcwd()))
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Output:

```
Before deletion: ['mydir1', 'main.py']
After deletion: ['main.py']
```

CONCLUDING REMARKS

In this chapter, we went through the ideas involved in Python file management. In addition, we spoke about the many different file modes that it may be opened in so that it can be processed. The processing of files, including reading, writing, deleting, renaming, and other operations, is demonstrated using a variety of examples and Python code. You should be able to develop your code for processing files based on these examples. In the next chapter, we will explore the notion of managing exceptions in Python and look at several instances.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:02.